

Análisis de Complejidad Temporal y Espacial

Proyecto: Sokoban — Tarea Integradora

Curso: Computación y Estructuras Discretas I

Equipo: Herrera · Fernández · Quintero

Convenciones

Símbolo	Significado
<code>n</code>	número de elementos en la estructura
<code>h</code>	altura del árbol BST
<code>b</code>	número de cajas en el tablero
<code>g</code>	número de objetivos (goals) en el tablero
<code>k</code>	longitud de una clave (string)

1. CustomLinkedList<T>

Lista enlazada simple con punteros `head` y `tail`. El puntero `tail` es lo que permite que `add` sea $O(1)$ sin necesidad de recorrer la lista completa.

Operaciones básicas

Método	Temporal	Espacial
<code>add(element)</code>	$O(1)$	$O(1)$
<code>remove(element)</code>	$O(n)$	$O(1)$
<code>get(index)</code>	$O(n)$	$O(1)$
<code>contains(element)</code>	$O(n)$	$O(1)$
<code>size() / isEmpty()</code>	$O(1)$	$O(1)$

Método	Temporal	Espacial
<code>clear()</code>	$O(1)^*$	$O(1)$

* `clear()` asigna `head = tail = null` en tiempo constante desde la perspectiva del programa. Los nodos quedan huérfanos y el Garbage Collector de Java los liberará en un momento posterior no determinista. La complejidad $O(1)$ aplica al costo que paga el programa, no a la liberación de memoria del sistema.

Merge Sort — `sort(Comparator<T>)`

Decidimos usar Merge Sort porque es el algoritmo de ordenamiento más natural para listas enlazadas: la operación de *merge* solo redirige punteros (no copia datos), lo que elimina la necesidad de memoria auxiliar proporcional a n que tendría el mismo algoritmo sobre arrays.

Implementación: recursiva con división por puntero lento/rápido (*tortoise-and-hare*).

Complejidad temporal: $O(n \log n)$

Hay $\log_2 n$ niveles de recursión. En cada nivel se realizan en total n comparaciones durante la fase de merge. El producto es $n \times \log n$.

Complejidad espacial: $O(\log n)$

Al ser recursivo, la pila de llamadas crece hasta la profundidad del árbol de recursión, que es $\log_2 n$. No se crean copias de los datos; solo se reordenan punteros entre nodos existentes. Si se implementara de forma iterativa (bottom-up), el espacio sería $O(1)$, pero se optó por la versión recursiva por claridad.

Búsqueda Binaria — `binarySearch(T target, Comparator<T>)`

Precondición: la lista debe estar ordenada con el mismo criterio que se usará en la búsqueda.

El algoritmo mantiene índices lógicos `[low, high]` y en cada iteración calcula `mid = low + (high - low) / 2`. Luego llama a `get(mid)` para recuperar el elemento central y decide si el objetivo está en la mitad izquierda o derecha.

Complejidad temporal: $O(n \log n)$

El algoritmo hace $O(\log n)$ iteraciones. Sin embargo, cada llamada a `get(mid)` recorre la lista desde `head` en $O(n)$, porque las listas enlazadas no tienen acceso aleatorio. El total es $O(n \times \log n)$.

Para comparación: en un array, `get(mid)` sería $O(1)$ y el total quedaría en $O(\log n)$. La diferencia de rendimiento es una consecuencia directa de que los arrays tienen memoria contigua y las listas no.

Complejidad espacial: $O(1)$

Solo se usan variables de control (`low`, `high`, `mid`). No se crea ninguna estructura auxiliar.

2. CustomStack<T>

Pila implementada con lista enlazada donde `top` apunta al elemento más reciente. En el juego almacena snapshots de estado (`GameSnapshot`) para el mecanismo de undo.

Método	Temporal	Espacial
<code>push(element)</code>	$O(1)$	$O(1)$
<code>pop()</code>	$O(1)$	$O(1)$
<code>peek()</code>	$O(1)$	$O(1)$
<code>isEmpty()</code> / <code>size()</code>	$O(1)$	$O(1)$
<code>clear()</code>	$O(1)^*$	$O(1)$

* Mismo caso que `clear()` en lista enlazada: $O(1)$ para el programa, liberación de memoria diferida al GC.

La pila completa ocupa $O(m)$ donde m = número de movimientos realizados desde que se cargó el nivel (un snapshot por movimiento).

3. CustomQueue<T>

Cola con punteros `front` (para dequeue) y `rear` (para enqueue). Ambas operaciones son $O(1)$ gracias a que no hay necesidad de recorrer la estructura en ningún caso. En el juego funciona como buffer de entrada para no perder teclas cuando el usuario presiona rápido.

Método	Temporal	Espacial
<code>enqueue(element)</code>	$O(1)$	$O(1)$
<code>dequeue()</code>	$O(1)$	$O(1)$
<code>peek()</code>	$O(1)$	$O(1)$
<code>isEmpty() / size()</code>	$O(1)$	$O(1)$
<code>clear()</code>	$O(1)^*$	$O(1)$

* Igual que en las estructuras anteriores.

4. CustomPriorityQueue<T>

Cola de prioridad mínima implementada como lista enlazada ordenada ascendentemente. El elemento mínimo siempre está en `head`.

El trade-off de esta implementación es deliberado: `insert` es costoso ($O(n)$) porque mantiene el orden durante la inserción, pero `extractMin` es $O(1)$ porque el mínimo siempre está al frente. Para el leaderboard del juego esto tiene sentido: se insertan pocas partidas por sesión, pero se consulta el mejor resultado con frecuencia.

Método	Temporal	Espacial
<code>insert(element)</code>	$O(n)$	$O(1)$
<code>extractMin()</code>	$O(1)$	$O(1)$
<code>peek()</code>	$O(1)$	$O(1)$
<code>isEmpty() / size()</code>	$O(1)$	$O(1)$
<code>clear()</code>	$O(1)^*$	$O(1)$

* Mismo comportamiento de GC que las demás estructuras enlazadas.

5. TranspositionTable

Tabla hash con encadenamiento separado (*separate chaining*). Almacena estados del tablero ya visitados durante la búsqueda automática, usando como clave la posición del jugador concatenada con las posiciones de todas las cajas.

Función de hash

$h = 0$

para cada carácter c en key :

$$h = (h \times 31 + c) \% \text{capacity}$$

retornar $|h|$

Hash polinomial de base 31, igual al que usa `String.hashCode()` internamente en Java (aunque Java usa base 31 también). La diferencia es que `String.hashCode()` de Java puede devolver valores negativos por desbordamiento de `int`, por eso aplicamos `Math.abs()` al resultado. El objetivo es distribuir las claves uniformemente entre los buckets.

Complejidad de la función: $O(k)$ donde k = longitud de la clave.

Operaciones

Las complejidades del promedio asumen un factor de carga bajo (por debajo de 0.75, que es el umbral de redimensionamiento). En el peor caso teórico, si todas las claves colisionan en el mismo bucket, la tabla degenera en una lista enlazada y las operaciones son $O(n)$.

Método	Temporal promedio	Temporal peor caso	Espacial
<code>put(key, value)</code>	$O(1)$ amortizado	$O(n)$	$O(1)$
<code>get(key)</code>	$O(1)$ amortizado	$O(n)$	$O(1)$
<code>containsKey(key)</code>	$O(1)$ amortizado	$O(n)$	$O(1)$
<code>remove(key)</code>	$O(1)$ amortizado	$O(n)$	$O(1)$
<code>clear()</code>	$O(\text{capacity})$	$O(\text{capacity})$	$O(1)$

El "amortizado" en las operaciones principales significa que el costo promedio sobre una secuencia larga de operaciones es $O(1)$, aunque operaciones individuales (como una que

dispara un resize) pueden costar $O(n)$. El resize ocurre $O(\log n)$ veces durante n inserciones, lo que distribuye ese costo y mantiene $O(1)$ amortizado por inserción.

Redimensionamiento

Se activa cuando $\text{size} / \text{capacity} \geq 0.75$. La nueva capacidad es $\text{capacity} \times 2 + 1$ (impar para acercarse a números primos y mejorar la distribución del hash).

	Complejidad
Temporal	$O(n)$ — rehash de todos los elementos
Espacial	$O(n)$ — nuevo arreglo del doble de tamaño

Espacio total de la tabla: $O(n + \text{capacity})$

6. BinarySearchTree<T>

Árbol binario de búsqueda donde cada nodo cumple: todos los del subárbol izquierdo son menores y todos los del derecho son mayores. Se usa para almacenar el historial de partidas (**Stats**) y obtenerlo ordenado por puntaje mediante recorrido in-order.

La altura h determina la complejidad de casi todas las operaciones. En un árbol balanceado $h = \log n$, pero si las inserciones llegan en orden estrictamente creciente o decreciente, el árbol degenera en una lista ($h = n$). Para el caso de partidas de juego las inserciones son en orden variable, por lo que en la práctica el árbol se mantiene razonablemente balanceado sin necesidad de rotaciones (AVL o rojo-negro).

Método	Temporal promedio	Temporal peor caso	Espacial
<code>insert(element)</code>	$O(\log n)$	$O(n)$	$O(h)$ pila recursiva
<code>search(element)</code>	$O(\log n)$	$O(n)$	$O(1)$ iterativo
<code>delete(element)</code>	$O(\log n)$	$O(n)$	$O(h)$ pila recursiva
<code>findMin()</code>	$O(\log n)$	$O(n)$	$O(1)$
<code>findMax()</code>	$O(\log n)$	$O(n)$	$O(1)$

Método	Temporal promedio	Temporal peor caso	Espacial
<code>size()</code> / <code>isEmpty()</code>	$O(1)$	$O(1)$	$O(1)$
<code>clear()</code>	$O(1)$	$O(1)$	$O(1)$

El caso $O(n)$ ocurre cuando el árbol está completamente degenerado. Por ejemplo, si se insertan los valores 1, 2, 3, 4, 5 en ese orden, cada nodo queda como hijo derecho del anterior y el árbol es equivalente a una lista enlazada.

Eliminación con dos hijos

Cuando el nodo a eliminar tiene dos hijos, se reemplaza su valor con el del **sucesor in-order** (el mínimo del subárbol derecho), y luego se elimina ese sucesor. Esto preserva la propiedad BST en $O(h)$.

Recorridos

Todos visitan cada nodo exactamente una vez.

Método	Orden	Temporal	Espacial
<code>inOrderTraversal()</code>	izq → raíz → der	$O(n)$	$O(n)$ resultado + $O(h)$ pila
<code>preOrderTraversal()</code>	raíz → izq → der	$O(n)$	$O(n) + O(h)$
<code>postOrderTraversal()</code>	izq → der → raíz	$O(n)$	$O(n) + O(h)$

Espacio total del árbol: $O(n)$

7. Motor del juego — Game.java

Método	Temporal	Espacial
<code>move(Direction)</code>	$O(b)$	$O(b)$
<code>isLevelComplete()</code>	$O(b \times g)$	$O(1)$

Método	Temporal	Espacial
<code>undo()</code>	$O(b)$	$O(1)$
<code>buildStateKey()</code>	$O(b)$	$O(b)$
<code>processInputBuffer()</code>	$O(n \times b)$	$O(1)$
<code>loadLevel(Level)</code>	$O(\text{rows} \times \text{cols})$	$O(b)$

`move` es $O(b)$ porque busca si hay una caja en la posición destino recorriendo la lista de cajas, y además guarda un snapshot con las b posiciones actuales. `isLevelComplete` es $O(b \times g)$ porque por cada caja verifica si alguno de los g objetivos coincide con su posición.

8. Tabla resumen

Estructura	Inserción	Búsqueda	Eliminación	Espacio
CustomLinkedList	$O(1)$ al final	$O(n)$ lineal	$O(n)$	$O(n)$
+ MergeSort (una vez)	$O(n \log n)$	—	—	$O(\log n)$ extra
+ BinarySearch (post-sort)	—	$O(n \log n)$	—	$O(1)$ extra
CustomStack	$O(1)$ push	$O(1)$ peek	$O(1)$ pop	$O(n)$
CustomQueue	$O(1)$ enqueue	$O(1)$ peek	$O(1)$ dequeue	$O(n)$
CustomPriorityQueue	$O(n)$ insert	$O(1)$ peek	$O(1)$ extractMin	$O(n)$
TranspositionTable	$O(1)$ amort.	$O(1)$ amort.	$O(1)$ amort.	$O(n + \text{cap})$
BinarySearchTree	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	$O(n)$
